

Technical Summary Hybrid IPS Berbasis Suricata + XGBoost

untuk Mitigasi Serangan SQL Injection

Sumber: Skripsi Ahmad Nabih Baril Hilmy (NIM: 225150207111085)
Universitas Brawijaya, Teknik Informatika, 2025

1. GAMBARAN UMUM SISTEM

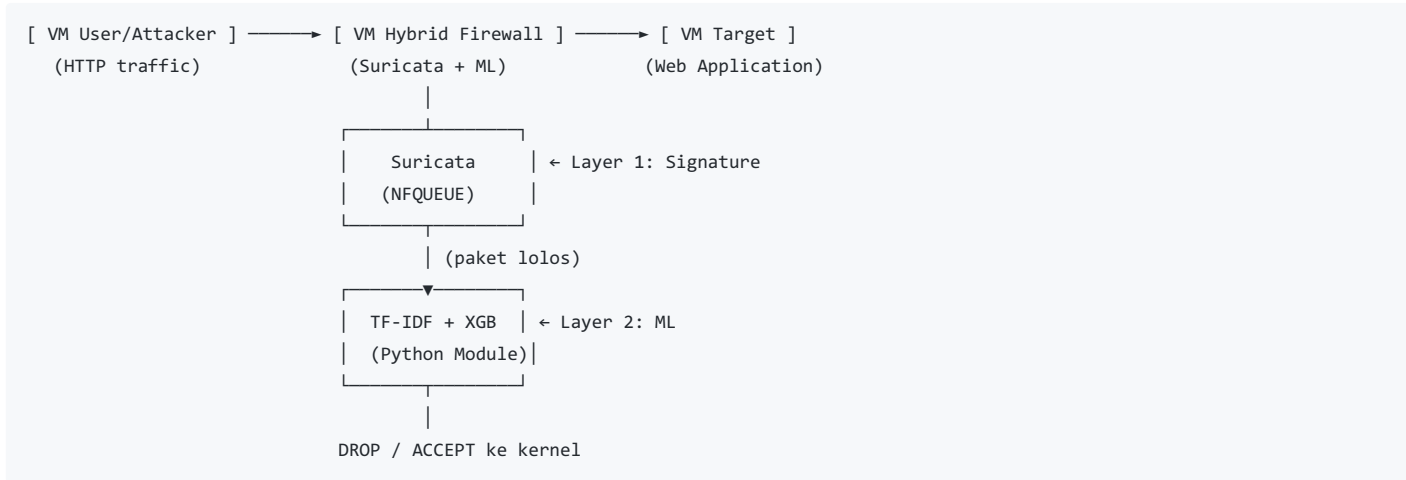
Sistem yang dibangun adalah **Hybrid Intrusion Prevention System (HIPS)** yang menggabungkan dua lapisan deteksi:

Lapisan	Teknologi	Metode	Posisi
Layer 1	Suricata	Signature-based detection	First-line filter
Layer 2	TF-IDF + XGBoost	ML-based classification	Second-line analysis
Penghubung	NFQUEUE (Netfilter)	Packet queue bridge	Antara Layer 1 & 2

Prinsip utama: Suricata menangkap serangan SQLi yang sudah dikenal (eksplisit). Paket yang lolos dari signature diteruskan ke XGBoost untuk analisis adaptif – menangkap serangan yang ter-obfuscate atau memiliki variasi sintaks baru.

2. TOPOLOGI & INFRASTRUKTUR

2.1 Arsitektur Jaringan



2.2 Komponen Virtual Machine

VM	Peran	Konten
VM User/Attacker	Sumber traffic	Mengirim HTTP request (normal + SQLi)
VM Hybrid Firewall	Titik inspeksi utama	Suricata, NFQUEUE, Python ML module
VM Target	Objek perlindungan	Web application yang rentan SQLi

Catatan: Seluruh traffic dari VM User **wajib** melewati VM Firewall terlebih dahulu, tidak boleh direct ke VM Target.

2.3 Keamanan Infrastruktur

- Protokol **WireGuard** digunakan untuk mengamankan jalur komunikasi antar VM
- Tujuan: meminimalkan noise dari sumber eksternal yang tidak terkontrol selama eksperimen

3. ALUR KERJA SISTEM (FLOW)

3.1 Alur Utama Paket



3.2 Mode Operasi Suricata

- Suricata berjalan dalam **mode inline (IPS)** melalui NFQUEUE
- Bukan sekedar IDS (monitor) – melainkan IPS (aktif blokir)
- Konfigurasi: `suricata.yaml` dengan pengaturan NFQUEUE

4. SPESIFIKASI KOMPONEN TEKNIS

4.1 Suricata (Layer 1)

Fungsi: First-line static filter untuk pola SQLi yang sudah dikenal

Konfigurasi kunci:

- Mode: inline via NFQUEUE
- Ruleset: 24 aturan **custom** (bukan ET Open atau bawaan Suricata)
- SID range: 1000001 hingga 1000024 (SID 1000022 tidak digunakan)

Buffer HTTP yang diperiksa:

Buffer	Digunakan untuk
http.uri	Parameter GET, query string
http.request_body	Payload POST
http.user_agent	Deteksi identitas Sqlmap

http.raw_uri Buffer	URL-encoded payload Digunakan untuk
http.method	Membedakan GET vs POST

Opsi aturan penting:

- nocase — pencocokan tidak case-sensitive
- pcre: "/.../" — Perl Compatible Regular Expression untuk pola kompleks
- content: "... " — pencocokan kata kunci sederhana
- flowbits:isnotset,alerted; flowbits:set,alerted; — cegah alert duplikat dalam satu flow
- flow:established,to_server — hanya periksa traffic dari client ke server
- classtype:web-application-attack

4.2 Ruleset Suricata — 24 Aturan Custom

Dikelompokkan dalam **5 klaster fungsional**:

Klaster 1: Reconnaissance & Fingerprinting (7 aturan)

SID	Deskripsi	Target Buffer
1000001	SQLMap User-Agent detection	http.user_agent
1000015	MySQL banner (@@version)	http.uri
1000016	PostgreSQL banner (version())	http.uri
1000017	SQL Server banner (@@SERVERNAME)	http.uri
1000018	Column enumeration via ORDER BY n	http.uri
1000020	Current user (CURRENT_USER() , USER() , SESSION_USER())	http.uri
1000021	Database info (DATABASE() , SCHEMA())	http.uri

Klaster 2: Syntactic & Basic Injection (3 aturan)

SID	Deskripsi	Target Buffer
1000002	Single/double quote & SQL comment markers	http.uri (PCRE)
1000003	Comment markers (-- , # , /*)	http.uri (PCRE)
1000004	Boolean blind — AND/OR kondisi GET	http.uri (PCRE)
1000005	Boolean blind — AND/OR kondisi POST	http.request_body (PCRE)

Klaster 3: Advanced & Blind Exploitation (8 aturan)

SID	Deskripsi	Target Buffer
1000006	SLEEP() — MySQL time-based (GET)	http.uri
1000007	SLEEP() — MySQL time-based (POST)	http.request_body
1000008	pg_sleep() — PostgreSQL time-based	http.uri
1000009	WAITFOR DELAY — SQL Server time-based	http.uri
1000010	UNION SELECT (GET)	http.uri
1000011	UNION SELECT (POST)	http.request_body

1000012 SID	EXTRACTVALUE() — error-based Deskripsi	http.uri Target Buffer
1000013	CONCAT + 0x7e — error-based	http.uri

Klaster 4: Obfuscation & Evasion (4 aturan)

SID	Deskripsi	Target Buffer
1000003	SQL comment markers	http.uri
1000014	URL-encoded chars (%27 , %22 , %23 , %2D%2D)	http.raw_uri
1000019	Null byte injection (%00)	http.uri
1000023	Hex-encoded SQL (0x[hex])	http.uri (PCRE)

Klaster 5: High-Risk System Access (1 aturan)

SID	Deskripsi	Target Buffer
1000024	LOAD_FILE() — file read attempt	http.uri

4.3 NFQUEUE (Penghubung)

Library: libnetfilter_queue (Netfilter Linux)

Fungsi teknis:

- Mengalihkan paket dari kernel space ke user space (Python module)
- Python module melakukan preprocessing + inferensi ML
- Hasil keputusan dikembalikan ke kernel: ACCEPT atau DROP

Mode Suricata yang diperlukan: queue / inline — paket berjalan langsung di jalur traffic (bukan mirror)

Alur NFQUEUE:

```
Kernel → NFQUEUE → Python script (TF-IDF + XGBoost) → Verdict (NF_ACCEPT / NF_DROP) → Kernel
```

4.4 TF-IDF (Feature Extraction)

Library: scikit-learn (TfidfVectorizer)

Input: Raw HTTP request string (query string / URL parameter)

Output: Sparse numerical vector (fitur untuk XGBoost)

Formula:

```
TF(t, d) = f(t,d) / total_terms_in_d

IDF(t) = log( N / (1 + df(t)) )

TF-IDF(t, d) = TF(t, d) × IDF(t)
```

Preprocessing sebelum TF-IDF:

1. Ekstraksi HTTP request (query string / payload)
2. Normalisasi komponen: host, path, non-query parameter distandarisasi
3. Hapus entri user-agent (untuk hindari bias)
4. Labeling: 1 = attack, 0 = benign
5. Vektorisasi TF-IDF
6. Gabungkan + shuffle dataset

Karakteristik token yang mendapat bobot tinggi:

- Kata kunci SQL: UNION, SELECT, SLEEP, EXTRACTVALUE, LOAD_FILE
- Operator logika: AND, OR, =
- Karakter khusus: ', ", --, #, /*
- Fungsi DBMS: @@version, DATABASE(), USER()

4.5 XGBoost (Classifier)

Library: xgboost Python package

Tipe: Supervised classification (binary: attack / benign)

Formula prediksi:

$$\hat{y}_i = \sum f_k(x_i) \quad \text{untuk } k = 1..K$$

di mana:

- $\hat{y}_i$ = prediksi sampel ke-i
- K = total pohon keputusan
- f_k = fungsi pohon ke-k
- x_i = vektor fitur hasil TF-IDF

Fungsi objektif:

$$L = \sum l(y_i, \hat{y}_i) + \sum \Omega(f_k)$$

$$\Omega(f_k) = \gamma T + (1/2)\lambda \sum w_j^2$$

- T = jumlah daun pada pohon
- w_j = bobot prediksi tiap daun
- γ, λ = parameter regularisasi (mencegah overfitting)

Hyperparameter yang di-tuning:

Parameter	Keterangan
learning_rate	Step size setiap iterasi boosting
max_depth	Kedalaman maksimum pohon
n_estimators	Jumlah total pohon keputusan
gamma (γ)	Minimum loss reduction untuk split
lambda (λ)	Regularisasi L2 pada bobot daun
scale_pos_weight	Penyeimbang kelas (untuk imbalanced data)

Output → Keputusan:

```
if P(attack | x_i) ≥ threshold → DROP (serangan SQLi)
if P(attack | x_i) < threshold → ACCEPT (traffic normal)
```

5. DATASET

5.1 Data Pelatihan (Training)

Jenis	Sumber	Keterangan
Attack	Sqlmap (auto-generated)	11 teknik serangan SQLi
Benign	CIC-IDS2017 (publik)	Traffic HTTP normal yang beragam

5.2 Jenis Serangan dalam Dataset Training

No	Teknik
1	Basic command Attack

2 No	Boolean-based Attack Teknik
3	Error-based Attack
4	Union-based Attack
5	Time-based Attack
6	Stacked query Attack
7	Multivariety level encoding Attack
8	SQLi tampering Attack (single tamper)
9	SQLi heavy tamper combo (multi tamper)
10	Random tamper + random user agent
11	Random user agent + delay

5.3 Data Pengujian (Testing)

Jenis	Sumber	Keterangan
Attack testing	Burp Suite (manual)	SQLi dataset – request manual
Benign testing	Burp Suite (manual)	Request HTTP normal – manual

Catatan: Testing dilakukan manual dalam 24 jam pada jaringan terisolasi.

5.4 Preprocessing Pipeline



6. FOKUS DETEKSI

Sesuai batasan masalah skripsi:

Aspek	Detail
Protokol	HTTP saja (bukan HTTPS – tidak ada TLS inspection)
Lokasi injeksi	Query string / URL parameter (GET & POST)
Fitur yang dianalisis	Isi query dan payload HTTP
Yang TIDAK dianalisis	Struktur paket layer rendah (TCP/IP header, dll.)
Lingkungan	Terkontrol (virtualisasi) – bukan produksi skala besar
Throughput/latency	Tidak diuji pada skala traffic komersial besar

7. SKENARIO PENGUJIAN

Tiga skenario dijalankan untuk perbandingan objektif:

Skenario	Komponen Aktif	Tujuan
S1	Suricata saja	Baseline: efektivitas signature murni
S2	XGBoost saja	Baseline: efektivitas ML murni
S3	Hybrid (Suricata + XGBoost)	Sistem utama: evaluasi peningkatan hybrid

8. METRIK EVALUASI

8.1 Confusion Matrix

	Prediksi Attack	Prediksi Normal
Aktual Attack	TP (True Positive)	FN (False Negative)
Aktual Normal	FP (False Positive)	TN (True Negative)

8.2 Formula Metrik

Precision	= TP / (TP + FP)
Recall	= TP / (TP + FN)
F1-Score	= 2 × (Precision × Recall) / (Precision + Recall)
Accuracy	= (TP + TN) / (TP + TN + FP + FN)
FPR	= FP / (FP + TN)

8.3 Interpretasi Metrik dalam Konteks IPS

Metrik	Artinya jika buruk	Dampak
Precision rendah	Banyak FP (normal diblokir)	Layanan legitimate terganggu
Recall rendah	Banyak FN (serangan lolos)	Aplikasi web tetap rentan
FPR tinggi	Banyak false alarm	Traffic sah otomatis terblokir (IPS bersifat aktif)

Metrik	Artinya jika buruk	Dampak
--------	--------------------	--------

9. TOOLS YANG DIGUNAKAN

Tool	Fungsi dalam Penelitian
Suricata	IPS engine, signature-based detection
Sqlmap	Generate attack traffic untuk training data
Burp Suite	Generate manual HTTP request untuk testing (Proxy, Repeater, Intruder)
WireGuard	Secure tunnel antar VM
Python	ML module (TF-IDF + XGBoost + NFQUEUE integration)
scikit-learn	TF-IDF vectorizer
xgboost	XGBoost classifier
libnetfilter_queue	Python binding untuk NFQUEUE
Hypervisor/VM	Isolasi lingkungan eksperimen (snapshot untuk reproducibility)
CIC-IDS2017	Dataset publik untuk benign training traffic

10. REFERENSI TEKNIS KUNCI

Untuk membangun ulang sistem, referensi berikut paling relevan:

Topik	Sumber
TF-IDF + XGBoost untuk SQLi (akurasi 99.37%)	Pansare dkk. (2025)
XGBoost untuk network intrusion detection	De Carvalho Bertoli dkk. (2021)
Suricata inline IPS mode + NFQUEUE	docs.suricata.io / OISF (2025)
NFQUEUE library	netfilter.org/projects/libnetfilter_queue
Suricata + KNN hybrid (referensi arsitektur)	Syamsuddin & Barukab (2022)
SQLi variations & techniques	Abdullayev & Chauhan (2023); Chen dkk. (2021)
Sqlmap exploitation tool	Q. Li dkk. (2024); B. Kalaiselvi dkk. (2025)

11. CATATAN PENTING UNTUK REBUILD

- SID 1000022 tidak ada** dalam ruleset — loncat dari 1000021 ke 1000023. memang dilewati karena sudah ter merge dengan aturan 21, jadi total hanya 23 aturan.
- Threshold XGBoost** tidak disebutkan nilainya secara eksplisit dalam skripsi — perlu ditentukan melalui eksperimen (biasanya 0.5, bisa di-tune).
- Suricata harus dijalankan dalam mode inline** (bukan passive/IDS) agar NFQUEUE berfungsi sebagai pemblokir aktif. Konfigurasi di `suricata.yaml`.
- Data training serangan** dihasilkan dari Sqlmap, bukan dataset publik SQLi — artinya model sangat tergantung pada pola Sqlmap. Perlu tambahan dataset manual untuk generalisasi lebih baik.
- Scope terbatas pada HTTP** — jika target menggunakan HTTPS, dibutuhkan SSL termination atau TLS inspection sebelum Suricata dapat membaca payload.
- Semua komponen berjalan di VM Firewall** — satu mesin yang menjalankan Suricata, NFQUEUE, dan Python ML module sekaligus. Perlu perhatian terhadap

resource usage.

7. **Flowbits dalam ruleset** menggunakan flag `alerted` — mencegah alert duplikat dalam satu TCP flow. Ini berpotensi menyebabkan paket berikutnya dalam satu session tidak diperiksa ulang.